Chapter 10 worked solutions

Incremental Decoding and KV Representations

This guide covers Exercises 1–4 and 9–12. It follows the sequence of steps in a causal decoder, counts the work and storage associated with the key–value cache, and works through the effect of absolute positions and score scaling. Exercises 5–8 are in the companion Jupyter notebook, which is also available as a PDF.

1. Follow the first two predictions

Treat each word as one token. The prompt is *the red*. Its two input positions are *the* at index 0 and *red* at index 1. During **prefill**, the decoder processes both positions in a single causal pass. It produces one row of logits, or scores over the vocabulary, per input position. The row at *the* is for the token following *the*. The row at *red* is for the token following the whole prompt. Use the **logits at index 1**, the row for *red*, to sample *car*.

Sampling a token gives us its ID. The next decoder call takes that ID as input, so the input is now *car* and its position in the sequence is **2**, even though it occupies the only input row of this call. Each layer projects its representation at position 2 into a query, a key, and a value. The layer has saved its own keys and values for *the* and *red* during prefill. It appends the key and value for *car* before evaluating the query at index 2.

Decoder call	Input positions in the call	Row used to sample	Positions that row may read	Sampled token
Prefill	0 (*the*), 1 (*red*)	1 (*red*)	0 and 1	*car*
First decoding step	2 (*car*)	2 (*car*)	0, 1, and 2	*stopped*

The query at index 2 compares against the three keys to determine the attention weights. It then uses those weights to mix the three corresponding values. Its read pattern includes index 2 because causal attention allows the current position as well as earlier ones.

After this step, each layer keeps **the projected keys and values for positions 0, 1, and 2** in its key–value cache. The next query will use them again to compare and mix information from these positions. The old queries have already produced their rows of attention output. *Stopped* enters the cache when the decoder consumes it as input at position 3 in the following call.

2. Count rows and comparisons

A prompt of length three has already been processed. The first subsequent input extends the prefix to four positions. The second extends it to five. Count the work of those two calls for one sequence, one layer, and one query head.

With full-prefix recomputation, a length-four input produces four query rows. The query at index 0 can read one key, the query at index 1 can read two keys, and the remaining rows can read three and four keys. The causal diagonal is included. There are $1 + 2 + 3 + 4 = 10$ allowed query–key pairs. At length five, the extra row reads five keys, giving $10 + 5 = 15$ pairs for that call.

With cached decoding, the first of these calls computes only the query at index 3. It reads the three prompt keys and the newly appended key at index 3, giving four pairs. The second call computes only the query at index 4. The cache now has four previous keys and the key at index 4, giving five pairs.

Input just processed	Full query rows	Cached query rows	Full allowed pairs	Cached allowed pairs
First new token, prefix length 4	4	1	$1+2+3+4=10$	4
Second new token, prefix length 5	5	1	$1+2+3+4+5=15$	5
Total after prefill	**9**	**2**	**25**	**9**

The cache makes old key projections available for reuse. Each new query has its own coordinates and must compare them with every key it is allowed to read. Hence one query row still contains several scores, and the number of scores in that row grows with the prefix. The table counts allowed pairs. A kernel that computes a full score grid before masking can also perform arithmetic for entries outside those allowed pairs.

3. Ask two questions of one history

Both heads read the key at the first position, $k_1 = (1, 0)$, and the key at the second position, $k_2 = (0, 1)$. The corresponding values are $v_1 = (2, 0)$ and $v_2 = (0, 1)$. A dot product compares the query with a key, and softmax converts the two scores into weights over the two positions.

The first query, $q_1 = (1, 0)$, has dot products $(1, 0)$ against the ordered keys. The second query, $q_2 = (0, 1)$, has dot products $(0, 1)$. Since each query and key has width two, scale both sets of scores by $1/\sqrt{2}$. Write $s = 1/\sqrt{2}$ and compute the larger softmax weight as

$$a = \frac{\exp(s)}{\exp(s) + \exp(0)} = \frac{\exp(s)}{1 + \exp(s)} \approx 0.669762.$$

The smaller weight is $1/(1 + \exp(s)) = 1 - a \approx 0.330238$. The two heads have the same denominator because each has one score of s and one score of zero. Their scores are arranged in opposite orders, so the weights favor different positions.

Head	Scaled scores	Softmax weights	Weighted sum of the values
1	$(s, 0)$	$(a, 1-a)$	$a(2,0) + (1-a)(0,1) = (2a, 1-a)$
2	$(0, s)$	$(1-a, a)$	$(1-a)(2,0) + a(0,1) = (2(1-a), a)$

The outputs are approximately $(1.339523, 0.330238)$ and $(0.660477, 0.669762)$. The shared cache supplies the same pair of values to both heads. Their separate queries assign different coefficients to those values, which produces a different mixture for each head.

4. Count a decoder's cache

For one sequence at one position in one layer, a single KV head stores a key of width 64 and a value of width 64. That gives $64 + 64 = 128$ scalars. At two bytes per scalar the pair takes 256 bytes. Both sequences store 1,024 positions, and each of the 12 layers stores its own vectors for every position. Thus one KV head across all sequences and layers accounts for

$$2 \times 1,024 \times 12 \times (64 + 64) \times 2 = 6,291,456 \text{ bytes} = 6 \text{ MiB}.$$

The first factor of two counts sequences, and the last factor of two is the number of bytes per scalar. Convert bytes to mebibytes (MiB) by dividing by $2^{20} = 1,048,576$. The key and value each contribute half of the bytes here because they have equal widths and use the same dtype.

Multiply the one-head total by the number of stored KV heads. The 12 query heads remain in all three cases. With equal-sized groups, divide 12 by the number of KV heads to find how many queries read each stored history.

KV heads	Total bytes for both sequences	Total MiB	Query heads per KV head
12	75,497,472	72	$12/12 = 1$
3	18,874,368	18	$12/3 = 4$
1	6,291,456	6	$12/1 = 12$

The totals describe the keys and values actually stored. Query heads need temporary query vectors during a step, and these have a separate memory cost.

9. Spend a cache budget (optional)

Keep the table's model dimensions of 24 layers, key and value widths of 64, and two bytes per scalar. Only the number of KV heads, sequences, or positions changes. The table gives 192 MiB for one 2,048-position sequence with 16 KV heads. At the same length, reducing the number of stored heads to four makes the cache one quarter as large. Reducing it to one makes the cache one sixteenth as large.

KV heads	Cache per sequence of 2,048 positions	Complete sequences in 192 MiB
4	$192(4/16) = 48$ MiB	$192/48 = 4$
1	$192(1/16) = 12$ MiB	$192/12 = 16$

For one sequence with four KV heads, count the storage for a single extra position across all layers. It takes

$$24 \times 4 \times (64 + 64) \times 2 = 24,576 \text{ bytes per position.}$$

The budget is $192 \times 2^{20} = 201,326,592$ bytes, which gives $201,326,592/24,576 = 8,192$ positions. The same result follows by multiplying 2,048 by four, since one original-length sequence with four KV heads consumes one quarter of the budget.

These capacities assume the full 192 MiB is available for the occupied KV arrays. A decoder must also support the chosen context length. If its limit is 2,048 positions, each sequence is capped there even when the arrays could hold more. Model weights, temporary tensors, and unused reserved slots draw from the device's memory separately.

10. Catch a position reset (optional)

RoPE rotates the two coordinates by an angle that depends on the sequence position. Rotating $(1, 0)$ by angle θ gives $(\cos\theta, \sin\theta)$ under the chapter's convention. Here each step advances the angle by $\pi/2$. The cached key is at position 0, so its angle is zero and it remains $(1, 0)$.

At the correct query position of 1, the query turns through $\pi/2$ and becomes $(0, 1)$. Its unscaled dot product with the stored key is

$$(0, 1) \cdot (1, 0) = 0.$$

If the decoder mistakes the new chunk's local index 0 for the query's sequence position, it rotates the query by zero and obtains $(1, 0)$. With the same key, the dot product becomes $(1, 0) \cdot (1, 0) = 1$. The reset changes the angle between the vectors from a quarter turn to zero. It therefore changes the similarity even though the stored key stays the same.

The decoder needs the position of each new input as well as the saved rows. In this example the cache starts at sequence position 0 and holds one position, so the next chunk starts at 1. The incoming query and key use that offset for their rotations before the query reads the history.

11. Choose the scale after absorption (optional)

In the expanded calculation, one score numerator adds two dot products. The content query compares four coordinates with an expanded content key. The rotary query compares two coordinates with a rotary key. This is equivalent to taking one dot product between the two vectors formed by concatenating their content and rotary parts. Those vectors have width $4 + 2 = 6$, so the expanded denominator is $\sqrt{6}$.

To see what changes after absorption, use row vectors. The latent cache row c has nine coordinates, and the content key up-projection W has shape 9×4. The expanded key is cW and has width four. If the content query is q^C, then

$$q^C(cW)^\top = (q^C W^\top)c^\top.$$

The transformed content query $q^C W^\top$ has **nine coordinates**, since it multiplies a row of width four by a 4×9 matrix. The order of multiplication has changed, while the scalar content score remains the same in exact arithmetic. The rotary term is also unchanged. Use $\sqrt{6}$ after absorption to preserve the original score and its softmax weights.

Using $\sqrt{d_c + d_R} = \sqrt{9 + 2} = \sqrt{11}$ would multiply every original scaled score by $\sqrt{6/11} \approx 0.738549$. This factor is positive, so it preserves score order. It reduces the gap between unequal scores, so softmax assigns less weight to the highest score and becomes less concentrated. For example, if the original two scores are $(1, 0)$, the first position gets weight $\exp(1)/(\exp(1) + 1) \approx 0.731059$. After the extra rescaling, the scores are approximately $(0.738549, 0)$ and the first position gets weight about 0.676678. If all scores are equal, their weights are already uniform and remain uniform under either scale.

12. Design an inference comparison (optional)

A decoder reads a prompt and predicts its first continuation token. When it receives that token as its next input, it can either rebuild the entire prefix or compute the new row and read the stored keys and values. Compare those two execution paths with the same weights, token IDs, masks, head grouping, and position convention. Use evaluation mode throughout so that operations such as dropout are disabled.

First check correctness in float64 on CPU with the six token IDs $[7, 8, 9, 10, 7, 16]$. Run one full causal pass to obtain the logits for positions 0 through 5. Then replay the same IDs using a cache and the chunk sizes $(3, 2, 1)$. Start each run with a fresh cache. The cache grows by one row per input position in every layer.

Call	Tokens supplied to the cached path	Absolute query positions	Key positions in the updated cache
Prefill	$[7, 8, 9]$	0, 1, 2	0, 1, 2
First append	$[10, 7]$	3, 4	0, 1, 2, 3, 4
Second append	$[16]$	5	0, 1, 2, 3, 4, 5

In the middle chunk, the query at position 3 must be blocked from reading position 4 even though both keys have been appended to the cache. Use the absolute positions in the mask and in any positional transformation such as RoPE. Compare every vocabulary logit at each cached output position with the corresponding row of the full pass, using absolute and relative tolerances of 10^{-10} for the float64 check. Also repeat with $(3, 1, 1, 1)$ to exercise the single-token path. Report the maximum absolute difference for each chunk. If a comparison fails, report the first differing position and inspect the corresponding mask row, position index, and cache length. Comparing only the most likely tokens could miss changes to the scores of other vocabulary entries.

For timing, choose one GPU, float16 inference, a batch of four prompts of length 512, and a replayed continuation of 128 token IDs per sequence. Use the same decoder in both paths, with 12 layers, 12 query heads, three KV heads, and key and value widths of 64. Choose a model that supports at least 640 positions. Use the same attention operations where the two paths permit it, and report the actual kernel in each phase. Project only the last hidden row of each timed call to vocabulary logits in both paths. Prepare the input tensors on the device and load the weights before starting either timer.

Phase	Full-prefix path	Cached path
Prompt	Process all 512 prompt tokens and produce the logits for the first continuation token.	Do the same prompt pass and populate the per-layer KV arrays. Include any allocation performed for the request.
Decode	Replay the first 127 continuation IDs one at a time. Each call runs the whole prefix, of length 513 up to 639, and returns the last position's logits.	Replay those same 127 IDs. Each call computes one new input row, appends its keys and values, reads the available history, and returns the last position's logits.

The prompt pass supplies the first prediction. Each subsequent call consumes one supplied token and produces the logits for the following one, giving 128 predictions in all. Both runs replay known IDs, which keeps the prefixes identical even if roundoff would change a sampling decision. The final replay ID is the target of the last prediction and does not need to be processed for this timing definition.

Warm up both paths on the same shapes. Time 20 paired trials and alternate which path runs first. Synchronize the GPU before and after each timed phase so the measurement includes the queued work. Measure the decode loop as a whole, then divide by 127 for milliseconds per decoding step at batch size four. Also report $4 \cdot 127$ divided by the decode time in seconds for continuation predictions per second after the prompt. Report medians and the interquartile spread for prompt and decode times separately.

The persistent cache size is the sum of the key and value payloads across all layers. For batch size B, layer count L, sequence length T, KV head count H_{KV}, key and value widths d_k and d_v, and s bytes per scalar, compute

$$M_{KV} = BLTH_{KV}(d_k + d_v)s.$$

Here $s = 2$. One stored position across the whole batch and all layers takes $4 \cdot 12 \cdot 3 \cdot (64 + 64) \cdot 2 = 36,864$ bytes. The logical cache contains 512 positions after the prompt, or 18 MiB. It contains 639 positions after the last timed decode call, or 23,556,096 bytes, approximately 22.465 MiB. If the implementation reserves space for 640 positions, report that allocated payload separately as 22.5 MiB. Full-prefix recomputation retains no KV arrays between calls, although it still creates keys and values as part of each forward pass.

Measure peak allocated memory for each phase in separate profiling runs. Start from a baseline with model weights and replay inputs already resident. Track live allocations for the persistent cache and temporary work at the same point in time. The temporary category includes attention workspace and any old and new buffers that coexist while growing a cache. Report the peak temporary allocation and the peak total separately, since the temporary peak can occur before the cache reaches its final size. The payload formula counts only the arrays. Allocation measurements also capture scratch buffers and implementation choices that affect how much device memory the run requires.